

Onboarding — Detección de productos: Clásico vs Deep Learning

Bienvenidos. Doc directa para que tengáis exactamente lo mismo que tenemos y podáis empezar a aportar. ~5-10 min de lectura + ~30 min de setup.

El proyecto en 30 segundos

ABP de Visión por Computador (UAB). Implementamos **dos pipelines paralelos de detección** sobre el MISMO dataset (un subset de MVTec D2S, 20 productos compatibles con Mercadona) y los comparamos:

- **Pipeline A — Visión clásica:** Selective Search + descriptores HOG y SIFT/BoVW + SVM χ^2 , con preprocesado handcrafted (CLAHE + white balance + denoise bilateral).
- **Pipeline B — Deep Learning:** YOLOv8s pre-entrenado en COCO, fine-tuned sobre nuestras clases.

El entregable académico no es ganar a YOLO con el clásico — es **cuantificar la brecha** y entender **dónde se rompe cada paradigma** (especialmente en triadas confusables: 4 manzanas, 3 pastas Reggia, 2 Coca-Cola...).

La regla fundamental

Para que la comparación sea válida, los dos pipelines comparten:

- el mismo dataset y los mismos splits train / val / test;
- las mismas 20 etiquetas con los mismos IDs (1 a 20);
- las mismas métricas (mAP@0.5, F1, IoU, matriz de confusión, tiempo);
- el mismo hardware para medir tiempos (CPU local + Colab T4).

Si tocáis algo de eso, decidlo en el grupo antes — un cambio unilateral invalida toda la comparativa.

Estado actual del proyecto

Hito	Estado	Qué
H1	✓ Hecho	EDA + selección de 20 clases verificadas en D2S
H2	✓ Hecho	Splits filtrados train/val/test en formato COCO
H3	✓ Hecho	Pipeline clásico (proposals + descriptores + codebook BoVW)
H4	✓ Hecho (código)	Entrenamiento + inferencia. Entreno full pendiente (vía Colab)
H5	✓ Hecho (código)	Hard negative mining
Preprocess	✓ Hecho	CLAHE + WB + denoise bilateral integrados y configurables
H6	Pendiente	YOLOv8s fine-tuned en Colab T4
H7	Pendiente	Framework evaluación común (mAP, F1, confusión)
H8	Pendiente	Análisis de robustez (ruido, blur, iluminación, oclusión)
H9	Opcional	Demo webcam
H10	Pendiente	Informe + presentación

Setup desde 0 (Windows + PowerShell)

1. Pre-requisitos

Instalar **Git for Windows** (git-scm.com) y **VSCode** (code.visualstudio.com). No hace falta Python aparte — uv lo gestiona.

2. Clonar el repo

Pedidle a Artur la URL del repositorio si no la tenéis ya. Ejecutar en PowerShell donde queráis el proyecto:

```
git clone <URL del repo>
cd grocery-tracker
```

3. Setup automático

Instala uv si falta y sincroniza dependencias (~5-15 min, baja PyTorch ~2 GB):

```
.\scripts\setup.ps1
```

4. Conseguir MVTec D2S

El dataset (~6 GB) **no está en git**. Dos opciones:

- Pedírselo a Artur (lo tiene extraído, os pasa la carpeta data/d2s).
- Bajarlo de <https://www.mvtec.com/company/research/datasets/mvtec-d2s> (rellenar formulario, aceptar licencia, descargar d2s_images_v*.tar.xz y d2s_annotations_v*.tar.xz, moverlos a data/raw/ y correr `uv run python scripts/prepare_d2s.py`).

Resultado deseado: data/d2s/images/ con ~21k .jpg y data/d2s/annotations/ con los .json de D2S.

5. Generar splits filtrados (rápido, ~10s)

Filtra D2S a las 20 clases y crea train/val/test estratificados con seed=42 (reproducible):

```
uv run python scripts/prepare_splits.py
```

6. Entrenar codebook BoVW (~1 min)

Vocabulario visual de K=300 palabras. Determinista con seed=42:

```
uv run python scripts/train_codebook.py
```

7. Smoke test del pipeline clásico (~2 min)

Si imprime Pipeline clasico funciona end-to-end sin crashes, vuestro entorno está OK:

```
uv run python scripts/test_classical_tiny.py
```

Mac / Linux

Mismo flujo. Solo cambia el setup: `bash scripts/setup.sh` en lugar del `.ps1`. Si trabajáis en Mac y al correr `scripts/run_classical_train.py` sklearn rompe por OOM, abrir `configs/classical.yaml` y verificar que `classifier.n_jobs: 1` y `classifier.sample_steps: 1` (ya están así por defecto — son los ajustes memoria-safe para Macs 8GB).

Notebooks recomendados (para empaparse del proyecto)

Notebook	Qué muestra
00_dataset_eda	El dataset MVTec D2S y validación de las 20 clases
01_class_selection	Cómo se construyen los splits + visualizaciones
02_classical_dev	Cada componente del clásico (SS, HOG, SIFT, BoVW)
03_training_visualization	Cómo se etiquetan los proposals (positivos / negativos)
04_classical_results	Detecciones del modelo (necesita entreno previo)
05_preprocessing_viz	CLAHE + WB + denoise visualmente

Tareas pendientes que podéis coger

Tarea	Qué hace falta
H6 — YOLOv8s en Colab	Cuenta Colab. Fine-tune YOLOv8s sobre los splits ya generados (formato YOLO se exp
H7 — Framework eval	Módulo nuevo en src/grocery_detection/eval/: mAP@0.5 y @0.5:0.95 (pycocotools), F1
H8 — Robustez	Generar test perturbado (ruido gaussiano $\sigma \in \{0.01, 0.05, 0.1\}$, blur $\sigma \in \{1, 3, 5\}$, brillo {
H10 — Informe	Estructura del documento académico + figuras / tablas auto-generadas. Cuando H6 + H

Sugerencia de reparto: uno coge H6 (YOLO en Colab), otro coge H7 (eval framework). H7 se puede empezar en paralelo a H6 — solo necesita los JSON, que podemos mockear inicialmente para probar la lógica.

Briefing técnico completo en PROYECTO.md del repo. Dudas: preguntar a Artur. **No tocar nada que afecte a splits, IDs de clase o métricas sin avisar.**